Método de regla falsa

Javier Alexander López Contreras
Facultad de Ciencias Físico Matemáticas (FCFM), Universidad Autónoma de Coahuila, Campus Campo redondo

Recibido el 24 de febrero del 2026

En este documento se analizará el método de regla falsa, el cual se usa para aproximar raíces de una función. También se busca el análisis y aplicación del código escrito en el lenguaje de programación Python, mediante el interprete Visual Studio Code.

1. INTRODUCCIÓN

El método de la **Regla Falsa** es un algoritmo numérico diseñado para encontrar raíces de funciones no lineales de la forma $f(x) = 0$. Este método usa un intervalo inicial $[a, b]$ para que cumpla la condición de **Bolzano** $(f(a) \cdot f(b) < 0$.

2. ALGORITMO Y EXPLICACIÓN DEL MÉTODO DE REGLA FALSA

El **método de la regla falsa** es similar al método de bisección, con la diferencia que en lugar de tomar el punto medio, se toma como nuevo valor la intersección con el eje x una linea recta formada por dos puntos del intervalo. Lo importante de este método es que genera aproximaciones y ofrece una prueba para asegurarse que la raíz quede entre dos iteraciones sucesivas.

Este método se usa por su capacidad de encontrar las raíces de una función más rápido que el método de bisección.

Al igual que en bisección se usa un primer intervalo $[a, b]$ y estos tienen que cumplir el teorema de Bolzano. El método consigue una aproximación mejor usando el punto $(c, 0)$ en el que la recta secante pasa por los puntos $(a, f(a))$ y $(b, f(b)$ cruza el eje x.

Para encontrar el valor de c, se realiza el siguiente algoritmo:

- Utilizamos la ecuación de la recta utilizando

$$y_2 - y_1 = m(x_2 - x_1) \tag{1}$$

- Tenemos nuestro punto $(a, f(a))$ y $(b, f(b))$ y evaluamos en el despeje de m de la ecuación anterior para obtener la pendiente de la recta.

$$m = \frac{f(b) - f(a)}{b - a} \tag{2}$$

- Hacemos lo mismo para el punto $(b, f(b))$ y $(c, 0)$

$$m = \frac{0 - f(b)}{c - b} \tag{3}$$

- Igualamos ambas pendientes.

$$\frac{f(b) - f(a)}{b - a} = \frac{0 - f(b)}{c - b} \tag{4}$$

- Despejamos c y así obtenemos su valor.

$$c = b - \frac{f(b)(b - a)}{f(b) - f(a)} \tag{5}$$

Si $f(a)$ y $f(c)$ tienen distinto signo, hay un cero en el punto $[a, c]$, si tienen el mismo signo, entonces el cero está en el punto $[c, b]$. Si $f(c) = 0$ entonces c es un cero de la función.

3. ANÁLISIS DEL CÓDIGO

El código se escribió en el lenguaje **Python**, el código usa una interfaz gráfica de usuario que facilita la interacción y visualización de los datos.

3.1. Configuración de la interfaz y entrada de datos

Para la captura de la función, se utiliza el objeto **Entry**. La evalucación se realiza mediante la función **eval**(), lo que permite que el motor de cálculo interprete funciones complejas importadas de **Numpy**.

```python
#empezar abriendo la ventana
#crear la ventana de tkinter
ventana = tk.Tk()
ventana.title('Porgrama para encontrar las ra ces de un polinomio mediante el m todo de regla falsa')
ventana.geometry("700x700")

#Aqui le pido al usuario que ingrese la funci n

def f(x):
    #usar las funciones importadas de numpy arriba
    return eval(funcion.get())

funcion=tk.Entry(ventana, width=50)
funcion.pack()
```

Listing 1: Configuración de la ventana y captura de función

3.2. Visualización gráfica

Se implementó una función de graficación utilizando **Matplotlib**. Esto es vital para que el usuario identifique visualmente el intervalo $[a, b]$ donde existe un cambio de signo, cumpliendo así con el **Teorema de Bolzano**.

```python
#Hacer la gr fica de la funci n
def grafica():
    #ajustar el rango de x para ver mejor la funcion
    x = np.linspace(-10, 10, 400)
    y = f(x)
    plt.plot(x,y, label='f(x)')
    plt.axhline(0, color='blue', lw=0.5)
    plt.axvline(0, color='blue', lw=0.5)
    plt.title(f'Gr fica de f(x) = {funcion.get()}')
    plt.xlabel('x')
    plt.ylabel('f(x)')
    plt.grid()
    plt.legend()
    plt.show()
```

Listing 2: Rutina de graficación con Matplotlib

3.3. Implementación del método de regla falsa

El núcleo del algoritmo traduce el fundamento matemático a un proceso iterativo. A diferencia de la bisección, este método calcula la aproximación c mediante la intersección de una línea recta con el eje x. El código también gestiona la cuantificación de errores para determinar la precisión de la aproximación actual.

$$E_{abs} = |c_{actual} - c_{viejo}| \tag{6}$$

$$E_{rel} = \frac{E_{abs}}{|c_{actual}|} \tag{7}$$

```python
    # Implementar el algoritmo de regla falsa aqu
    while iter_count <= max_iter:
        #aplicar formula de la posicion falsa
        c = b - (f(b)*(b-a)) / (f(b)-f(a))

        if iter_count > 1:
            error_absoluto = abs(c-c_viejo)
            error_relativo = error_absoluto / abs(c) if c != 0 else float('inf')

            err_abs_str = f"{error_absoluto:.6e}"
            err_rel_str = f"{error_relativo:.6e}"

        else:
            err_abs_str = '---'
            err_rel_str = '---'

        tabla.insert('', tk.END, values= (iter_count, f"{c:.6e}", err_abs_str, err_rel_str))

        #si ya estamos cerca de la raiz nos salimos
        if abs(f(c)) < tol:
            resultado.config(text=f"La ra z encontrada es: {c:.6e}", fg='green')
            break

        if f(c) * f(a) < 0:
            b = c
        else:
            a = c

        #actualizar el c_viejo para el error del siguiente ciclo
        c_viejo = c
        iter_count += 1
```

Listing 3: Algoritmo de Regla Falsa y cálculo de errores

4. RESULTADOS Y ANÁLISIS

En esta etapa se evaluó el desempeño del software frente a funciones con comportamientos específicos para poner a prueba la precisión del algoritmo y la interfaz gráfica desarrollada.

4.1. Análisis de convergencia: Caso $x^x - 100$

Se realizó una prueba utilizando la función $f(x) = x^x - 100$, partiendo de un intervalo $[3, 4]$ con la tolerancia de 10^{-6}. A diferencia de bisección, el método de regla falsa utiliza la pendiente de la secante para .ªpuntar"más cerca de la raíz real en cada paso. El programa logró encontrar la raíz aproximada $x = 3.597258$ en menor iteraciones que el método de bisección, debido a que aprovecha los valores $f(a)$ y $f(b)$ para ajustar la posición de c.

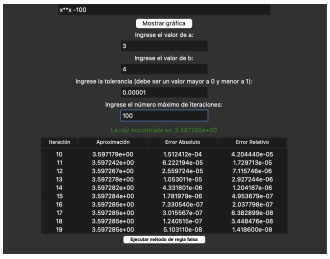

Figura 1: Prueba 1 del método de regla falsa

4.2. Prueba con función trigonométrica: $f(x) = sin(x)$

Usando el intervalo $[3, 4]$ se busca el valor de π. Aunque existen infinitas raíces para la función seno, el programa rige estrictamente por el cambio de signo de Bolzano en el intervalo seleccionado, atrapando solo el valor de π

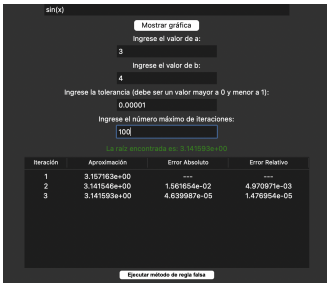

Figura 2: Prueba con la función seno

4.3. Prueba de función con asíntotas

Este un caso de error. La función cruza el eje x, pero en $x = 2$ hay una discontinuidad, no hay raíz. Es importante observar que el programa al alcanzar el max_iter o si arroja un error de división por cero al intentar calcular la pendiente de la secante.

Figura 3: Prueba con asíntotas

5. CONCLUSIONES

Se puede concluir que el método de regla falsa es una herramienta superior en comparación con el método de bisección, esto debido a su rapidez para encontrar las raíces. También es muy fiable al momento de encontrar la raíz, pero esto depende de la correcta selección del rango y de que la función sea continua. Al usar una cierta tolerancia protege al sistema de infinitas iteraciones y nos otorga resultados reproducibles.

6. Referencias

[1] Chapra, S. C., y Canale, R. P. (2015). *Métodos numéricos para ingenieros* (7a ed.). México: McGraw-Hill.

[2] Nieves, A., y Domínguez, F. C. (2014). *Métodos numéricos aplicados a la ingeniería* (4a ed.). México: Grupo Editorial Patria.

[3] Python Software Foundation. (2026). *The Python Tutorial: Floating Point Arithmetic, Issues and Limitations*. Recuperado de https://docs.python.org/3/tutorial/floatingpoint.html